

WHEN DO PROMPTING AND PREFIX-TUNING WORK? A THEORY OF CAPABILITIES AND LIMITATIONS

Aleksandar Petrov, Philip H.S. Torr & Adel Bibi

Department of Engineering Science

University of Oxford

Oxford, United Kingdom

{aleksandar.petrov, philip.torr, adel.bibi}@eng.ox.ac.uk

ABSTRACT

Context-based fine-tuning methods, including prompting, in-context learning, soft prompting (also known as prompt tuning), and prefix-tuning, have gained popularity due to their ability to often match the performance of full fine-tuning with a fraction of the parameters. Despite their empirical successes, there is little theoretical understanding of how these techniques influence the internal computation of the model and their expressiveness limitations. We show that despite the continuous embedding space being more expressive than the discrete token space, soft-prompting and prefix-tuning are potentially less expressive than full fine-tuning, even with the same number of learnable parameters. Concretely, context-based fine-tuning cannot change the relative attention pattern over the content and can only bias the outputs of an attention layer in a fixed direction. This suggests that while techniques like prompting, in-context learning, soft prompting, and prefix-tuning can effectively elicit skills present in the pretrained model, they may not be able to learn novel tasks that require new attention patterns.

1 INTRODUCTION

Language model advances are largely driven by larger models and more training data (Kaplan et al., 2020; Rae et al., 2021). Training cutting-edge models is out of reach for most academic researchers, small enterprises, and individuals, and it has become common to instead fine-tune open-source pre-trained models (Devlin et al., 2019; Min et al., 2021). Yet, due to escalating computational demands, even fine-tuning of the larger models has become prohibitively expensive (Lialin et al., 2023).

As a result, there is an acute need for more efficient fine-tuning methods, either by sparsely modifying the parameters of the model or modifying its input context. Examples of the first type include adapter modules which introduce a few trainable layers to modify the behaviour of the frozen pre-trained network (Rebuffi et al., 2017; Houlsby et al., 2019; Hu et al., 2023). One can also use low-rank updates, which also results in a reduced number of trainable parameters (Hu et al., 2021).

Context-based fine-tuning has been motivated by the success of few-shot and zero-shot learning (Wei et al., 2021; Kojima et al., 2022). The most popular context-based approach is prompting, where generation is conditioned on either human-crafted or automatically optimized tokens (Shin et al., 2020; Liu et al., 2023). In-context learning —prompting via providing input-label examples— is another widely used technique (Brown et al., 2020). Given the challenges of discrete optimization over tokens, there is a growing interest in methods that optimize real-valued embeddings (Lester et al., 2021). It is widely believed that these *soft prompts* offer greater expressiveness due to the expansive nature of continuous space. Furthermore, beyond only optimizing input embeddings, one can optimize the inputs of every attention layer (Li and Liang, 2021). This technique, *prefix-tuning*, has proven to be very successful and competitive to full fine-tuning (Liu et al., 2022).

While context-based fine-tuning approaches have witnessed impressive empirical successes and widespread adoption, we have little theoretical understanding of how they work. In this work, we analyse the influence of prompts and prefixes on the internal computations of a pretrained model and delineate their limitations. Specifically, we address the following questions:

1. **Soft prompting and prefix-tuning are motivated by the embedding space being larger than the token space. However, can a transformer utilize the additional capacity?** We show that with a careful choice of transformer weights, controlling a single embedding can generate any of the V^N completions of N tokens, while controlling a token can produce only V completions, with V being the vocabulary size. Thus, a transformer can indeed exploit the embedding space.
2. **Since prefix-tuning is more expressive than prompting, is it as expressive as full fine-tuning?** Despite the expressiveness of continuous space, prefix-tuning has structural limitations. A prefix cannot change the relative attention over the content tokens and can only bias the output of the attention block in a constant direction. In contrast, full fine-tuning can learn new attention patterns and arbitrarily modify attention block outputs, making it strictly more powerful.
3. **If context-based fine-tuning methods suffer from such structural limitations, how come they have high empirical performance?** We show that the prefix-induced bias can steer the model towards a pretraining task. Prefix-tuning can also combine skills picked up during pretraining to solve some new tasks similar to pretraining tasks. However, it may not learn a completely new task. This is not simply because of the small number of learnable parameters: fine-tuning the same number of parameters can be sufficient to learn the novel task. Hence, context-based fine-tuning can elicit or combine pretrained model skills but cannot learn completely new behaviors.

2 BACKGROUND

2.1 THE TRANSFORMER ARCHITECTURE

We outline a simplified decoder-only transformer architecture (Vaswani et al., 2017). Assume that the model has vocabulary size V (also referred to as *number of tokens*). The input is a sequence $(\mathbf{x}_1, \dots, \mathbf{x}_p)$, $\mathbf{x}_i \in \{1, \dots, V\}$, $\forall i$. Each token is mapped to a d_e -dimensional vector that is the $\mathbf{x}_i$ -th column of an embedding matrix $\mathbf{E} \in \mathbb{R}^{d_e \times V}$. The attention mechanism is position-invariant, so typically position encodings are added. For a model with maximum input length N (*context size*), we use a one-hot position encoding $\mathbf{e}_N(i)$ concatenated with the embedding. Therefore, the embedding for the i -th position provided to the first attention block would be $\mathbf{x}_i = [\mathbf{E}_{:, \mathbf{x}_i}^\top, \mathbf{e}_N^\top(i)]^\top$.

A transformer consists of alternating attention blocks which operate across the whole sequence and Multi-Layer Perceptrons (MLPs) that operate on each individual element. Each attention block consists of H heads. Each head h is parameterized by query, key, and value matrices $\mathbf{W}_Q^h, \mathbf{W}_K^h \in \mathbb{R}^{k \times d_{in}}, \mathbf{W}_V^h \in \mathbb{R}^{d_{out} \times d_{in}}$.¹ The attention matrix $\mathbf{A}^h \in \mathbb{R}^{p \times p}$ for head h then has elements

$$\mathbf{A}_{ij}^h = \frac{\exp\left(\frac{T}{\sqrt{k}}(\mathbf{W}_Q^h \mathbf{x}_i)^\top (\mathbf{W}_K^h \mathbf{x}_j)\right)}{\sum_{r=1}^p \exp\left(\frac{T}{\sqrt{k}}(\mathbf{W}_Q^h \mathbf{x}_i)^\top (\mathbf{W}_K^h \mathbf{x}_r)\right)}, \quad (1)$$

where $p \leq N$ is the current length of the input and $T > 0$ is an inverse temperature parameter.² Equation (1) is the softmax function, hence with high enough T , it will result in approximately one-hot encoding of the maximum j . The output of the attention block $\mathcal{A}$ with H heads is then $(\mathbf{t}_1, \dots, \mathbf{t}_p)$, where each position i is the sum of the attention-weighted values across all heads:

$$\begin{aligned} \mathcal{A}[(\mathbf{W}_Q^1, \dots, \mathbf{W}_Q^H), (\mathbf{W}_K^1, \dots, \mathbf{W}_K^H), (\mathbf{W}_V^1, \dots, \mathbf{W}_V^H)](\mathbf{x}_1, \dots, \mathbf{x}_p) &= (\mathbf{t}_1, \dots, \mathbf{t}_p), \\ \mathbf{t}_i &= \sum_{h=1}^H \sum_{j=1}^p \mathbf{A}_{ij}^h \mathbf{W}_V^h \mathbf{x}_j. \end{aligned} \quad (2)$$

A transformer then applies an MLP to each output of an attention block before passing them to next attention block. We will consider linear layers $\mathcal{L}[\mathbf{M}, \mathbf{b}](\mathbf{x}) = \mathbf{M}\mathbf{x} + \mathbf{b}$ and ReLU-activated linear layers $\hat{\mathcal{L}}[\mathbf{M}, \mathbf{b}](\mathbf{x}) = \text{ReLU}(\mathbf{M}\mathbf{x} + \mathbf{b})$. When we compose attention blocks and linear or softmax layers, we will implicitly assume that the linear layer is applied to all positions of the sequence. Furthermore, we will use the *then* operator $\circ$ for left-to-right function composition. Therefore, a transformer model predicting confidences over the vocabulary can, for example, be represented as:

$$(\mathbf{y}_1, \dots, \mathbf{y}_p) = \left(\mathcal{A}_1 \circ \hat{\mathcal{L}}_{1,1} \circ \mathcal{L}_{1,2} \circ \mathcal{A}_2 \circ \hat{\mathcal{L}}_{2,1} \circ \mathcal{L}_{2,2} \circ \text{softmax} \right) \left(\left[\begin{array}{c} \mathbf{E}_{:, \mathbf{x}_1} \\ \mathbf{e}_N(1) \end{array} \right], \dots, \left[\begin{array}{c} \mathbf{E}_{:, \mathbf{x}_p} \\ \mathbf{e}_N(p) \end{array} \right] \right), \quad (3)$$

¹For the first block, d_{in} must be $d_e + N$ but may be different for the deeper blocks.

²A causal model has $\mathbf{A}_{ij} = 0$ for $j > i$. This does not affect our results so we will skip the masking step.

where the output dimension of the last layer has to be V . The next token for a deterministic transformer is selected to be the last element's largest logit: $x_{p+1} = \arg \max_{u \in 1, \dots, V} y_{p,u}$. Given an input $(x_1, \dots, x_p)$, the model then autoregressively extends this sequence one token at a time, following Equation (3) either until the sequence reaches a length N or until a special termination token.

A transformer has no separation between the system prompt S , user provided input X and the autoregressively response Y . Thus, a sequence conditional on user input is denoted as $(S_1, \dots, S_{n_S}, X_1, \dots, X_{n_X}, Y_1, \dots, Y_{n_Y})$ and one without user input as $(S_1, \dots, S_{n_S}, Y_1, \dots, Y_{n_Y})$.

2.2 CONTEXT-BASED FINE-TUNING OF A PRETRAINED MODEL

We now define prompting, soft prompting and prefix-tuning with the previously introduced notation.

Prompting. The most frequently used content-based fine-tuning approach is *prompting*: prefixing the input $(X_1, \dots, X_{n_X})$ with a token sequence $S \in \{1, \dots, V\}^{n_S}$ to guide the model response: $(S_1, \dots, S_{n_S}, X_1, \dots, X_{n_X})$. This is how most people interact with language models such as ChatGPT.

Soft prompting. Soft prompting replaces the embeddings of the system input $E_{:,s_i}$ with learned vectors $s_i \in \mathbb{R}^{d_e}$ called *virtual tokens* (Hambardzumyan et al., 2021; Lester et al., 2021; Qin and Eisner, 2021). Hence, the input in Equation (3) is modified to be:

$$\left(\begin{bmatrix} s_1 \\ e_N(1) \end{bmatrix}, \dots, \begin{bmatrix} s_{n_S} \\ e_N(n_S) \end{bmatrix}, \begin{bmatrix} E_{:,X_1} \\ e_N(n_S + 1) \end{bmatrix}, \dots, \begin{bmatrix} E_{:,X_{n_X}} \\ e_N(n_S + n_X) \end{bmatrix} \right) \quad (4)$$

with s_i chosen to maximize the likelihood of a target response $Y=(Y_1, \dots, Y_{n_Y})$, i.e., $\arg \max_{s_1, \dots, s_{n_S} \in \mathbb{R}^{d_e}} \sum_{j=1}^{n_Y} \log y_{n_S+n_X+j, Y_j}$, where $y_{n_S+n_X+j}$ are autoregressively generated.

Prefix-tuning. Prefix-tuning applies soft prompting across the depth of the model (Li and Liang, 2021; Liu et al., 2022). The first n_S positions for all attention blocks are learnable parameters, replacing the input $(x_1^l, \dots, x_{n_X}^l)$ for layer l with $(s_1^l, \dots, s_{n_S}^l, x_1^l, \dots, x_{n_X}^l)$, where all s_i^l constitute the prefix. Hence, prefix-tuning can be formulated as $\arg \max_{\{s_1^l, \dots, s_{n_S}^l\}_{l=1}^{n_S}} \sum_{j=1}^{n_Y} \log y_{n_S+n_X+j, Y_j}$. Prefix-tuning has been successful at fine-tuning models (Vu et al., 2022; Wu and Shi, 2022; Choi and Lee, 2023; Ouyang et al., 2023; Bai et al., 2023), leading to calls for language models provided as a service (La Malfa et al., 2023) to allow providing prefixes instead of prompts (Sun et al., 2022).

Any token-based prompt $(S_1, \dots, S_{n_S})$ has a corresponding soft prompt $(s_i = E_{:,S_i})$ but the reverse does not hold. Similarly, every soft prompt $(s_1, \dots, s_{n_S})$ can be represented as a prefix by setting the deeper prefixes to be the values that the model would compute at these positions ($s_i^l = (\mathcal{A}_1 \circ \dots \circ \mathcal{L}_{l-1,-1})([s_1^T, e_N^T(1)]^T, \dots, [s_l^T, e_N^T(l)]^T)$). The reverse also does not hold: there are prefixes that cannot be represented as a soft prompt. A hierarchy emerges: *prompting* < *soft prompting* < *prefix-tuning*, with prefix-tuning the most powerful of the three. Hence, we focus on examining its performance relative to full fine-tuning but our findings also apply to prompting and soft prompting.

3 SOFT PROMPTING HAS MORE CAPACITY THAN PROMPTING

The success of soft prompting (and prefix-tuning) is commonly attributed to the larger capacity of the continuous embeddings compared to the finite tokens. Yet, increased capacity is beneficial only if the model can utilize it. We show this is indeed the case by constructing a transformer generating exponentially more completions by varying a single virtual token than by varying a hard token.

Consider unconditional generation (representing a function with no inputs) with a single system token: $(Y_1, \dots, Y_N) = f(S_1) = f_{S_1}$. For a deterministic autoregressive function, there are a total of V functions in this family, hence the upper bound on the number of outputs of length N that one can generate by varying the first token S_1 is V : the first token fully determines the rest of the sequence. Generally, if one varies the first N_S tokens, there are at most V^{N_S} unique outputs. What if instead of the token S_1 we vary a single virtual token s_1 : $(Y_1, \dots, Y_N) = f(s_1) = f_{s_1}$? This family of functions is indexed by a real vector and hence is infinite: in principle, one could generate all V^N possible